POLITECHNIKA ŚLĄSKA

Wydział Matematyki Stosowanej

Gliwice

SPRAWOZDANIE

z przedmiotu

Aplikacje dla systemu Android

Implementacja modelu obiektowego w Kotlinie

Laboratorium 3.

Autor: Bartosz Bieniek

Numer albumu: 315945

Grupa: 1

Prowadzący: dr hab. in. Adam Zielonka, prof. PŚ

Data wykonania: 24.10.2026

Gliwice, 2026

Spis treści

1. Cel ćwiczenia. 2
2. Wykorzystane technologie. 2
3. Model obiektowy. 2
   3.1. Modele domenowe. 2
   3.2. Identyfikatory. 3
   3.3. Repozytoria. 3
   3.4. Klasy rezultatów 7
   3.5. Implementacja księgozbioru 8
4. Prezentacja działania aplikacji. 11
   4.1. Zarządzanie biblioteką. 11
      4.1.1. Dodawanie książek. 11
      4.1.2. Rejestrowanie czytelników. 12
      4.1.3. Wyszukiwanie książek po tytule lub autorze, bez rozróżniania wielkości liter. 12
   4.2. Wypożyczanie książek. 12
      4.2.1. Wypożyczenie dostępnej książki przez zarejestrowanego czytelnika. 12
      4.2.2. Odmowa wypożyczenia książki, która jest już wypożyczona. 13
      4.2.3. Ograniczenie liczby aktywnych wypożyczeń jednego czytelnika. 13
   4.3. Zwracanie książek. 14
      4.3.1. Rejestrowanie zwrotu książki. 14
      4.3.2. Odmowa zwrotu niewypożyczonej książki. 14
   4.4. Historia. 14
      4.4.1. Wyświetlanie aktywnego wypożyczenia dla książki. 14
      4.4.2. Wyświetlanie aktywnych wypożyczeń dla czytelnika. 15
      4.4.3. Wyświetlanie wszystkich dokonanych wypożyczeń. 15
      4.4.4. Wyświetlanie wszystkich wypożyczeń dla książki. 15
      4.4.5. Wyświetlanie wszystkich wypożyczeń dla czytelnika. 16
5. Literatura. 16

1. Cel ćwiczenia.

Celem ćwiczenia było zaprojektowanie i zaimplementowanie prostego programu imitującego system biblioteczny. Aplikacja powinna umożliwiać zarządzanie książkami, czytelnikami oraz wypożyczeniami, zapisując dane w pamięci operacyjnej.

2. Wykorzystane technologie.

- Język *Kotlin* w wersji *2.4.20*
- Narzędzie *Gradle* w wersji *9.6.0*

3. Model obiektowy.

Aby odwzorować działanie systemu bibliotecznego, w pierwszej kolejności konieczne było utworzenie obiektów domenowych reprezentujące najważniejsze obiekty z perspektywy biznesowej - książki, czytelnika oraz wypożyczenia.

3.1. Modele domenowe.

W tym celu zdecydowałem się skorzystać z `data class`-ów, które pomogą mi zachować spójność danych. Są one z założenia niezmienne (niemutowalne), dzięki czemu możliwe jest uniknięcie sytuacji, w których obiekt po zapisaniu w kolekcji zostałby zmieniony z zewnątrz.

```
data class Book(
    val id: BookId = BookId(value = Uuid.random()),
    val title: String,
    val author: String,
)

data class Reader(
    val id: ReaderId = ReaderId(value = Uuid.random()),
    val firstName: String,
    val lastName: String,
)

data class Rental(
    val id: RentalId = RentalId(value = Uuid.random()),
    val book: Book,
    val reader: Reader,
    val rentedAt: Instant,
    val returnedAt: Instant? = null
) {
    init {
        check((returnedAt == null) || (rentedAt < returnedAt)) {
            "The return date must be after the rental date."
        }
    }

    val active: Boolean
        get() = returnedAt == null
}
```

Listing 1: Modele domenowe.

Model książki zawiera podstawowe informacje o lekturze: tytuł, autora oraz identyfikator, dzięki któremu możliwe będzie rozpoznanie konkretnej kopii. Model czytelnika zawiera podstawowe informacje, takie jak imię, nazwisko oraz identyfikator - możliwe jest przecież, aby z biblioteki mogło korzystać wiele tak samo nazywających się osób. Model wypożyczenia oprócz identyfikatora, książki, czytelnika i daty wypożyczenia, posiada też opcjonalne pole z datą zwrotu, która zostanie uzupełniona w momencie oddania książki. Gdy wartość zostanie wskazana, warto sprawdzić, czy jest ona poprawna, a więc czy następuje po dacie wypożyczenia. Do tego celu wykorzystałem blok inicjalizujący oraz instrukcję check, która wyrzuca wyjątek `IllegalStateException` w przypadku, gdy wskazany warunek logiczny nie zostanie spełniony.

3.2. Identyfikatory.

Pracując z wieloma obiektami korzystającymi z identyfikatorów tego samego typu (na przykład `String`, czy `Uuid`) łatwo o pomyłkę - może zdarzyć się, że przez nieostrożność będziemy wyszukiwać książki po identyfikatorze odczytanym z czytelnika. Można temu zaradzić tworząc bardzo lekkie `value class`-y, które „owijają" wskazaną wartość. Dzięki temu powstaje nowy typ, który kompilator jest w stanie sprawdzić statycznie w czasie kompilacji, a co za tym idzie, w wymienionej wyżej sytuacji zostanie zgłoszony błąd (jak porównać typy `BookId` i `ReaderId`).

```
@JvmInline
value class BookId(val value: Uuid)

@JvmInline
value class ReaderId(val value: Uuid)

@JvmInline
value class RentalId(val value: Uuid)
```

Listing 2: Identyfikatory obiektów zaimplementowane z wykorzystaniem `value class`-ów.

Warto zaznaczyć, że obiekty stworzone na podstawie `value class` przechowywane są przez wartość na stosie, a nie przez referencję na kopcu.

3.3. Repozytoria.

Aplikacja będzie musiała zawierać klasę realizującą mechanizm wypożyczeń, jednakże dla zwiększenia elastyczności i ułatwienia procesu testowania rozwiązania, można wykorzystać wzorzec repozytoriów. Polega on na odwróceniu zależności - klasa uzyskująca dostęp do danych korzysta z faktycznej implementacji przez interfejs.

Pierwszym krokiem było więc utworzenie interfejsów, przez które główna klasa będzie prowadziła odczyt i zapis danych.

Teoretycznie możliwe jest, żeby w trakcie działania programu powstała kopia obiektu domenowego zawierająca użyty już wcześniej identyfikator oraz zupełnie inne dane. Przyjąłem, że będę to traktować jako edycję obiektu, a co za tym idzie, chciałbym, żeby repozytoria funkcjonowały poprawnie mimo niepasujących (nowych) danych. Spowodowało to, że wiele metod realizuje odczyt danych jedynie na podstawie identyfikatora.

```
interface BookRepository {
    fun insert(book: Book)
    fun getAll(): List<Book>
    fun getById(bookId: BookId): Book?

    fun existsById(id: BookId): Boolean
    fun findAllByQuery(query: String): List<Book>
}

interface ReaderRepository {
    fun insert(reader: Reader)
    fun getAll(): List<Reader>
    fun getById(readerId: ReaderId): Reader?

    fun existsById(id: ReaderId): Boolean
}

interface RentalRepository {
    fun insert(rental: Rental)
    fun update(rental: Rental)
    fun getAll(): List<Rental>

    fun findAllForBookId(bookId: BookId): List<Rental>
    fun findActiveForBookId(bookId: BookId): Rental?
    fun findAllForReaderId(readerId: ReaderId): List<Rental>
    fun findActiveForReaderId(readerId: ReaderId): List<Rental>
    fun countActiveForReaderId(readerId: ReaderId): Int
}
```

Listing 3: Interfejsy repozytoriów.

Ponieważ nie chciałem mieszać odpowiedzialności ani powielać kodu, podjąłem decyzję o stworzeniu implementacji nieposiadających żadnej logiki biznesowej, a jedynie najprostszą walidację zapobiegającą niespójności danych.

Zgodnie z założeniami projektu, aplikacja powinna zapisywać dane wyłącznie w pamięci ulotnej, dlatego mój kod będzie wewnętrznie pracował na kolekcjach oferowanych przez język - głównie na słownikach (`Map`-ach). Zapewniają one błyskawiczny dostęp do danych oraz ułatwiają wyszukiwanie obiektów.

```
class InMemoryBookRepository(private val locale: Locale = Locale.ROOT) :
BookRepository {
    private val books = mutableMapOf<BookId, Book>()

    override fun insert(book: Book) {
        require(book.id !in books) {
            "A book with the same identifier already exists."
        }
        books[book.id] = book
    }

    override fun getAll(): List<Book> {
        return books.values.toList()
    }

    override fun getById(bookId: BookId): Book? {
        return books[bookId]
    }

    override fun existsById(id: BookId): Boolean {
        return id in books
    }

    override fun findAllByQuery(query: String) = books.values.filter { book ->
        book.title.containsIgnoreCase(query, locale) ||
book.author.containsIgnoreCase(query, locale)
    }

    private fun String.containsIgnoreCase(other: String, locale: Locale =
Locale.ROOT): Boolean {
        return lowercase(locale).contains(other.lowercase(locale))
    }
}
```

Listing 4: Implementacja repozytorium przechowującego książki.

```
class InMemoryReaderRepository : ReaderRepository {
    private val readers = mutableMapOf<ReaderId, Reader>()

    override fun insert(reader: Reader) {
        require(reader.id !in readers) {
            "A reader with the same identifier already exists."
        }
        readers[reader.id] = reader
    }

    override fun getAll(): List<Reader> {
        return readers.values.toList()
    }

    override fun getById(readerId: ReaderId): Reader? {
        return readers[readerId]
    }

    override fun existsById(id: ReaderId): Boolean {
        return id in readers
    }
}
```

Listing 5: Implementacja repozytorium przechowującego czytelników.

```
class InMemoryRentalRepository : RentalRepository {
    private val rentals = mutableMapOf<RentalId, Rental>()

    override fun insert(rental: Rental) {
        require(rental.id !in rentals) {
            "A rental with the same identifier already exists."
        }
        rentals[rental.id] = rental
    }

    override fun update(rental: Rental) {
        require(rental.id in rentals) {
            "A rental with the specified identifier does not exist."
        }
        rentals[rental.id] = rental
    }

    override fun getAll(): List<Rental> {
        return rentals.values.toList()
    }

    override fun findAllForBookId(bookId: BookId) = rentals.values.filter {
        it.book.id == bookId
    }

    override fun findActiveForBookId(bookId: BookId) =
rentals.values.firstOrNull {
        it.active && it.book.id == bookId
    }

    override fun findAllForReaderId(readerId: ReaderId) =
rentals.values.filter {
        it.reader.id == readerId
    }

    override fun findActiveForReaderId(readerId: ReaderId) =
rentals.values.filter {
        it.active && it.reader.id == readerId
    }

    override fun countActiveForReaderId(readerId: ReaderId) =
rentals.values.count {
        it.active && it.reader.id == readerId
    }
}
```

Listing 6: Implementacja repozytorium przechowującego wypożyczenia.

3.4. Klasy rezultatów

Mechanizm wypożyczeń powinien zwracać informacje o niepowodzeniu danych akcji. Ponieważ próba zwrotu niezarejestrowanej książki, czy wypożyczenia więcej niż maksymalnej ich

ilości nie jest sytuacją wyjątkową (na tyle żeby wyrzucać wyjątki), zdecydowałem się stworzyć interfejs `Result`. Będzie on zawierał dwa warianty (klasy), reprezentujące sukces oraz porażkę.

Do tego zadania idealnie sprawdzi się `sealed interface`, ponieważ uniemożliwi tworzenie kolejnych implementacji, poza tymi, które zostały zdefiniowane w tym samym pliku. Takie podejście upraszcza kod, na przykład znosząc wymóg posiadania w bloku `when` gałęzi `else` (oczywiście gdy dodamy ścieżki dla `Success` i `Failure`).

```
sealed interface LibraryError

sealed interface Result<out S, out F : LibraryError> {
    data class Success<out S>(val value: S) : Result<S, Nothing>
    data class Failure<out F : LibraryError>(val failure: F) : Result<Nothing,
F>
}
```

Listing 7: Definicja klasy rezultatów.

Warto zwrócić uwagę na oznaczenie w interfejsie parametrów generycznych słówkiem `out`. Informuje ono kompilator, że dany typ będzie wykorzystany jedynie do zwrotu danych, a co za tym idzie możliwa jest generalizacja typu - na przykład przypisanie wartości `Result<Int, LibraryEmptyError>` do zmiennej typu `Result<Number, LibraryError>`.

Na potrzeby implementacji księgozbioru, utworzyłem także odpowiednią hierarchię błędów.

```
sealed interface BookError : LibraryError {
    val book: Book
}

sealed interface ReaderError : LibraryError {
    val reader: Reader
}

data class BookNotRegisteredError(override val book: Book) : BookError
data class BookNotRentedError(override val book: Book) : BookError
data class BookAlreadyRentedError(override val book: Book) : BookError
data class ReaderNotRegisteredError(override val reader: Reader) : ReaderError
data class TooManyActiveRentalsForReaderError(override val reader: Reader) :
ReaderError
```

Listing 8: Hierarchia błędów.

3.5. Implementacja księgozbioru

Korzystając ze wszystkich wcześniej utworzonych implementacji, możliwe jest zaprogramowanie klasy reprezentującej księgozbiór, zawierającej mechanizmy związane z tym procesem.

Niektóre z metod i getterów jedynie przekazują dalej dane otrzymane przez parametry, inne natomiast, takie jak `rentBook` i `returnBook` dodatkowo weryfikują warunki biznesowe i modyfikują dane.

Warto zwrócić uwagę na przekazanie do klasy `Library` instancji zegara `Clock`. Ułatwia to znacząco testowanie automatyczne - na czas testów wystarczy wstrzyknąć inny zegar i modyfikować wirtualny czas, zamiast oczekiwać na wybicie właściwej godziny.

```
class Library(
    private val bookRepository: BookRepository,
    private val readerRepository: ReaderRepository,
    private val rentalRepository: RentalRepository,
    private val clock: Clock = Clock.System,
) {
    val books: List<Book>
        get() = bookRepository.getAll()

    val readers: List<Reader>
        get() = readerRepository.getAll()

    val rentals: List<Rental>
        get() = rentalRepository.getAll()

    fun registerBook(book: Book) {
        bookRepository.insert(book)
    }

    fun registerReader(reader: Reader) {
        readerRepository.insert(reader)
    }

    fun searchBooks(query: String): List<Book> {
        return bookRepository.findAllByQuery(query)
    }

    fun rentBook(book: Book, reader: Reader): Result<Rental, LibraryError> {
        if (!bookRepository.existsById(book.id)) {
            return Result.Failure(BookNotRegisteredError(book))
        }
        if (!readerRepository.existsById(reader.id)) {
            return Result.Failure(ReaderNotRegisteredError(reader))
        }
        if (rentalRepository.findActiveForBookId(book.id) != null) {
            return Result.Failure(BookAlreadyRentedError(book))
        }
        if (rentalRepository.countActiveForReaderId(reader.id) >=
MAXIMUM_ACTIVE_RENTALS_PER_READER) {
            return Result.Failure(TooManyActiveRentalsForReaderError(reader))
        }

        val book = bookRepository.getById(book.id)
        val reader = readerRepository.getById(reader.id)

        check(book != null && reader != null)

        val rental = Rental(book = book, reader = reader, rentedAt =
clock.now())

        rentalRepository.insert(rental)
```

```
        return Result.Success(rental)
    }

    fun returnBook(book: Book): Result<Rental, BookError> {
        if (!bookRepository.existsById(book.id)) {
            return Result.Failure(BookNotRegisteredError(book))
        }

        val originalRental = rentalRepository.findActiveForBookId(book.id)
            ?: return Result.Failure(BookNotRentedError(book))

        val updatedRental = originalRental.copy(returnedAt = clock.now())

        rentalRepository.update(updatedRental)
        return Result.Success(updatedRental)
    }

    fun getActiveRental(book: Book): Rental? {
        return rentalRepository.findActiveForBookId(book.id)
    }

    fun getActiveRentals(reader: Reader): List<Rental> {
        return rentalRepository.findActiveForReaderId(reader.id).sortedBy
{ it.rentedAt }
    }

    fun getRentalHistory(book: Book): List<Rental> {
        return rentalRepository.findAllForBookId(book.id).sortedBy
{ it.rentedAt }
    }

    fun getRentalHistory(reader: Reader): List<Rental> {
        return rentalRepository.findAllForReaderId(reader.id).sortedBy
{ it.rentedAt }
    }

    companion object {
        private const val MAXIMUM_ACTIVE_RENTALS_PER_READER = 3
    }
}
```

Listing 9: Implementacja księgozbioru.

Moja implementacja odwołuje się w dużej mierze do identyfikatorów, jednakże obiekty typu `Rental` zawierają migawki konkretnych czytelników i książek. Umożliwia to śledzenie ewentualnych zmian, które mogły nastąpić w ramach konta danego użytkownika (na przykład zmiana nazwiska). Aby zapewnić spójność danych i uniemożliwić utworzenie wypożyczenia z nieaktualnymi danymi, przed zapisem w metodzie `rentBook` pobierane są z repozytorium najświeższe dane.

4. Prezentacja działania aplikacji.

Przed przetestowaniem programu konieczne jest utworzenie obiektu księgozbioru, wraz ze wszystkimi niezbędnymi repozytoriami.

```
val locale = Locale.ROOT

val library = Library(
    bookRepository = InMemoryBookRepository(locale = locale),
    readerRepository = InMemoryReaderRepository(),
    rentalRepository = InMemoryRentalRepository(),
    clock = Clock.System,
)
```

Listing 10: Utworzenie obiektu biblioteki.

4.1. Zarządzanie biblioteką.

4.1.1. Dodawanie książek.

```
val a = Book(title = "Animal Farm", author = "George Orwell")
val b = a.copy(id = BookId(value = Uuid.random())) // Kolejny egzemplarz tej
samej książki.
val c = Book(title = "Fables for Robots", author = "Stanisław Lem")
val d = Book(title = "1984", author = "George Orwell")

println("\nRegistering books...")
library.registerBook(a)
library.registerBook(b)
library.registerBook(c)
// Ostatnia książka nie została na razie celowo zarejestrowana.

println("Registered ${library.books.size} books.")
println(library.books)
```

Listing 11: Utworzenie i dodanie książek.

```
Registering books...
Registered 3 books.
[Book(id=BookId(value=00389979-811f-406a-991e-1442db023d5e), title=Animal Farm, author=George Orwell),
 Book(id=BookId(value=33486aa7-503a-4114-80c5-860edaa6da95), title=Animal Farm, author=George Orwell),
 Book(id=BookId(value=3e796d70-be9c-4b8f-aba0-8655bc6eb82f), title=Fables for Robots, author=Stanisław Lem)]
```

Rysunek 1: Utworzenie i dodanie książek.

4.1.2. Rejestrowanie czytelników.

```
val p = Reader(firstName = "Toby", lastName = "Jones")
val q = Reader(firstName = "Martha", lastName = "Smith")
val r = Reader(firstName = "Jacob", lastName = "Smith")
val s = Reader(firstName = "Hans", lastName = "Jackson")

println("\nRegistering readers...")
library.registerReader(p)
library.registerReader(q)
library.registerReader(r)
// Ostatni czytelnik nie został na razie celowo zarejestrowany.

println("Registered ${library.readers.size} readers.")
println(library.readers)
```

Listing 12: Utworzenie i zarejestrowanie użytkowników.

```
Registering readers...
Registered 3 readers.
[Reader(id=ReaderId(value=bb61cd00-05f9-4006-a458-9a93ae33d133), firstName=Toby, lastName=Jones), Reader
 (id=ReaderId(value=c20a3711-f7e7-4b9b-ad7b-9f5aa01abce1), firstName=Martha, lastName=Smith), Reader
 (id=ReaderId(value=6f38112e-04b3-4e4d-bca5-141605147094), firstName=Jacob, lastName=Smith)]
```

Rysunek 2: Utworzenie i zarejestrowanie użytkowników.

4.1.3. Wyszukiwanie książek po tytule lub autorze, bez rozróżniania wielkości liter.

```
println("\nSearching books for phrase 'ani'...")
println(library.searchBooks("ani"))
println("\nSearching books for phrase 'anis'...")
println(library.searchBooks("anis"))
```

Listing 13: Wyszukiwanie książek po tytule i autorze.

```
Searching books for phrase 'ani'...
[Book(id=BookId(value=00389979-811f-406a-991e-1442db023d5e), title=Animal Farm, author=George Orwell),
 Book(id=BookId(value=33486aa7-503a-4114-80c5-860edaa6da95), title=Animal Farm, author=George Orwell),
 Book(id=BookId(value=3e796d70-be9c-4b8f-aba0-8655bc6eb82f), title=Fables for Robots, author=Stanisław Lem)]

Searching books for phrase 'anis'...
[Book(id=BookId(value=3e796d70-be9c-4b8f-aba0-8655bc6eb82f), title=Fables for Robots, author=Stanisław Lem)]
```

Rysunek 3: Wyszukiwanie książek po tytule i autorze.

4.2. Wypożyczanie książek.

4.2.1. Wypożyczenie dostępnej książki przez zarejestrowanego czytelnika.

```
println("\nRenting book a for user p...")
println(library.rentBook(a, p)) // Ok
println("\nRenting book d for user p...")
println(library.rentBook(d, p)) // Błąd: Książka D nie jest zarejestrowana.
println("\nRenting book b for user s...")
println(library.rentBook(b, s)) // Błąd: Czytelnik S nie jest zarejestrowany.
```

Listing 14: Wypożyczanie dostępnych książek przez zarejestrowanych użytkowników.

```
Renting book a for user p...
Success(value=Rental(id=RentalId(value=c8cb86c5-cd90-444d-bfa6-f3d5f3ddebcc), book=Book(id=BookId
 (value=00389979-811f-406a-991e-1442db023d5e), title=Animal Farm, author=George Orwell), reader=Reader
 (id=ReaderId(value=bb61cd00-05f9-4006-a458-9a93ae33d133), firstName=Toby, lastName=Jones),
 rentedAt=2026-10-25T14:28:42.474431Z, returnedAt=null))

Renting book d for user p...
Failure(failure=BookNotRegisteredError(book=Book(id=BookId(value=0a0e2134-0602-490f-8e42-6bb8e164793b),
 title=1984, author=George Orwell)))

Renting book b for user s...
Failure(failure=ReaderNotRegisteredError(reader=Reader(id=ReaderId
 (value=79cbdfa4-d069-438f-bf29-7f4024cf8e1f), firstName=Hans, lastName=Jackson)))
```

Rysunek 4: Wypożyczanie dostępnych książek przez zarejestrowanych użytkowników.

4.2.2. Odmowa wypożyczenia książki, która jest już wypożyczona.

```
println("\nRenting book a for user q...")
println(library.rentBook(a, q)) // Błąd: Książka A jest już wypożyczona.
```

Listing 15: Odmowa wypożyczenia pożyczonej książki.

```
Renting book a for user q...
Failure(failure=BookAlreadyRentedError(book=Book(id=BookId(value=00389979-811f-406a-991e-1442db023d5e),
 title=Animal Farm, author=George Orwell)))
```

Rysunek 5: Odmowa wypożyczenia pożyczonej książki.

4.2.3. Ograniczenie liczby aktywnych wypożyczeń jednego czytelnika.

Każdy użytkownik może mieć do trzech aktywnych wypożyczeń.

```
println("\nRenting three books for user q...")
library.registerBook(d) // Rejestrujemy czwartą książkę.
println(library.rentBook(b, p)) // Ok
println(library.rentBook(c, p)) // Ok
println(library.rentBook(d, p)) // Błąd: Użytkownik wypożyczył już 3 książki.
```

Listing 16: Ograniczenie liczby wypożyczonych książek dla czytelnika.

```
Renting three books for user q...
Success(value=Rental(id=RentalId(value=3bce5419-33c3-481e-8a2d-a920df99a860), book=Book(id=BookId
 (value=33486aa7-503a-4114-80c5-860edaa6da95), title=Animal Farm, author=George Orwell), reader=Reader
 (id=ReaderId(value=bb61cd00-05f9-4006-a458-9a93ae33d133), firstName=Toby, lastName=Jones),
 rentedAt=2026-10-25T14:28:42.478857Z, returnedAt=null))
Success(value=Rental(id=RentalId(value=7e452a9f-51f2-4434-b4f8-bcb790641c77), book=Book(id=BookId
 (value=3e796d70-be9c-4b8f-aba0-8655bc6eb82f), title=Fables for Robots, author=Stanisław Lem), reader=Reader
 (id=ReaderId(value=bb61cd00-05f9-4006-a458-9a93ae33d133), firstName=Toby, lastName=Jones),
 rentedAt=2026-10-25T14:28:42.478919Z, returnedAt=null))
Failure(failure=TooManyActiveRentalsForReaderError(reader=Reader(id=ReaderId
 (value=bb61cd00-05f9-4006-a458-9a93ae33d133), firstName=Toby, lastName=Jones)))
```

Rysunek 6: Ograniczenie liczby wypożyczonych książek dla czytelnika.

4.3. Zwracanie książek.

4.3.1. Rejestrowanie zwrotu książki.

```
println("\nReturning book a...")
println(library.returnBook(a)) // Ok
```

Listing 17: Rejestrowanie zwrotu książki.

```
Returning book a...
Success(value=Rental(id=RentalId(value=c8cb86c5-cd90-444d-bfa6-f3d5f3ddebcc), book=Book(id=BookId
 (value=00389979-811f-406a-991e-1442db023d5e), title=Animal Farm, author=George Orwell), reader=Reader
 (id=ReaderId(value=bb61cd00-05f9-4006-a458-9a93ae33d133), firstName=Toby, lastName=Jones),
 rentedAt=2026-10-25T14:28:42.474431Z, returnedAt=2026-10-25T14:28:42.479237Z))
```

Rysunek 7: Rejestrowanie zwrotu książki.

4.3.2. Odmowa zwrotu niewypożyczonej książki.

```
println("\nReturning book a...")
println(library.returnBook(a)) // Błąd: Książka nie jest wypożyczona.
```

Listing 18: Odmowa zwrotu niewypożyczonej książki.

```
Returning book a...
Failure(failure=BookNotRentedError(book=Book(id=BookId(value=00389979-811f-406a-991e-1442db023d5e),
 title=Animal Farm, author=George Orwell)))
```

Rysunek 8: Odmowa zwrotu niewypożyczonej książki.

4.4. Historia.

4.4.1. Wyświetlanie aktywnego wypożyczenia dla książki.

```
println("\nGetting active rentals for book a...")
println(library.getActiveRental(a))
println("\nGetting active rentals for book b...")
println(library.getActiveRental(b))
```

Listing 19: Wyświetlanie aktywnego wypożyczenia dla książki.

```
Getting active rentals for book a...
null

Getting active rentals for book b...
Rental(id=RentalId(value=3bce5419-33c3-481e-8a2d-a920df99a860), book=Book(id=BookId
 (value=33486aa7-503a-4114-80c5-860edaa6da95), title=Animal Farm, author=George Orwell), reader=Reader
 (id=ReaderId(value=bb61cd00-05f9-4006-a458-9a93ae33d133), firstName=Toby, lastName=Jones),
 rentedAt=2026-10-25T14:28:42.478857Z, returnedAt=null)
```

Rysunek 9: Wyświetlanie aktywnego wypożyczenia dla książki.

4.4.2. Wyświetlanie aktywnych wypożyczeń dla czytelnika.

```
println("\nGetting active rentals for reader p...")
println(library.getActiveRentals(p))
println("\nGetting active rentals for reader q...")
println(library.getActiveRentals(q))
```

Listing 20: Wyświetlanie aktywnych wypożyczeń dla czytelnika.

```
Getting active rentals for reader p...
[Rental(id=RentalId(value=3bce5419-33c3-481e-8a2d-a920df99a860), book=Book(id=BookId
 (value=33486aa7-503a-4114-80c5-860edaa6da95), title=Animal Farm, author=George Orwell), reader=Reader
 (id=ReaderId(value=bb61cd00-05f9-4006-a458-9a93ae33d133), firstName=Toby, lastName=Jones),
 rentedAt=2026-10-25T14:28:42.478857Z, returnedAt=null), Rental(id=RentalId
 (value=7e452a9f-51f2-4434-b4f8-bcb790641c77), book=Book(id=BookId
 (value=3e796d70-be9c-4b8f-aba0-8655bc6eb82f), title=Fables for Robots, author=Stanisław Lem), reader=Reader
 (id=ReaderId(value=bb61cd00-05f9-4006-a458-9a93ae33d133), firstName=Toby, lastName=Jones),
 rentedAt=2026-10-25T14:28:42.478919Z, returnedAt=null)]

Getting active rentals for reader q...
[]
```

Rysunek 10: Wyświetlanie aktywnych wypożyczeń dla czytelnika.

4.4.3. Wyświetlanie wszystkich dokonanych wypożyczeń.

```
println("\nGetting whole rental history...")
println(library.rentals)
```

Listing 21: Wyświetlanie wszystkich dokonanych wypożyczeń.

```
Getting whole rental history...
[Rental(id=RentalId(value=c8cb86c5-cd90-444d-bfa6-f3d5f3ddebcc), book=Book(id=BookId
 (value=00389979-811f-406a-991e-1442db023d5e), title=Animal Farm, author=George Orwell), reader=Reader
 (id=ReaderId(value=bb61cd00-05f9-4006-a458-9a93ae33d133), firstName=Toby, lastName=Jones),
 rentedAt=2026-10-25T14:28:42.474431Z, returnedAt=2026-10-25T14:28:42.479237Z), Rental(id=RentalId
 (value=3bce5419-33c3-481e-8a2d-a920df99a860), book=Book(id=BookId
 (value=33486aa7-503a-4114-80c5-860edaa6da95), title=Animal Farm, author=George Orwell), reader=Reader
 (id=ReaderId(value=bb61cd00-05f9-4006-a458-9a93ae33d133), firstName=Toby, lastName=Jones),
 rentedAt=2026-10-25T14:28:42.478857Z, returnedAt=null), Rental(id=RentalId
 (value=7e452a9f-51f2-4434-b4f8-bcb790641c77), book=Book(id=BookId
 (value=3e796d70-be9c-4b8f-aba0-8655bc6eb82f), title=Fables for Robots, author=Stanisław Lem), reader=Reader
 (id=ReaderId(value=bb61cd00-05f9-4006-a458-9a93ae33d133), firstName=Toby, lastName=Jones),
 rentedAt=2026-10-25T14:28:42.478919Z, returnedAt=null)]
```

Rysunek 11: Wyświetlanie wszystkich dokonanych wypożyczeń.

Warto zwrócić uwagę, że zakończone wypożyczenia posiadają ustawioną końcowa datę (`returnedAt`).

4.4.4. Wyświetlanie wszystkich wypożyczeń dla książki.

```
println("\nGetting rental history for book a...")
  println(library.getRentalHistory(a))
```

Listing 22: Wyświetlanie wszystkich wypożyczeń dla książki.

```
Getting rental history for book a...
[Rental(id=RentalId(value=c8cb86c5-cd90-444d-bfa6-f3d5f3ddebcc), book=Book(id=BookId
 (value=00389979-811f-406a-991e-1442db023d5e), title=Animal Farm, author=George Orwell), reader=Reader
 (id=ReaderId(value=bb61cd00-05f9-4006-a458-9a93ae33d133), firstName=Toby, lastName=Jones),
 rentedAt=2026-10-25T14:28:42.474431Z, returnedAt=2026-10-25T14:28:42.479237Z)]
```

Rysunek 12: Wyświetlanie wszystkich wypożyczeń dla książki.

4.4.5. Wyświetlanie wszystkich wypożyczeń dla czytelnika.

```
println("\nGetting rental history for reader p...")
  println(library.getRentalHistory(p))
```

Listing 23: Wyświetlanie wszystkich wypożyczeń dla czytelnika.

```
Getting rental history for reader p...
[Rental(id=RentalId(value=c8cb86c5-cd90-444d-bfa6-f3d5f3ddebcc), book=Book(id=BookId
 (value=00389979-811f-406a-991e-1442db023d5e), title=Animal Farm, author=George Orwell), reader=Reader
 (id=ReaderId(value=bb61cd00-05f9-4006-a458-9a93ae33d133), firstName=Toby, lastName=Jones),
 rentedAt=2026-10-25T14:28:42.474431Z, returnedAt=2026-10-25T14:28:42.479237Z), Rental(id=RentalId
 (value=3bce5419-33c3-481e-8a2d-a920df99a860), book=Book(id=BookId
 (value=33486aa7-503a-4114-80c5-860edaa6da95), title=Animal Farm, author=George Orwell), reader=Reader
 (id=ReaderId(value=bb61cd00-05f9-4006-a458-9a93ae33d133), firstName=Toby, lastName=Jones),
 rentedAt=2026-10-25T14:28:42.478857Z, returnedAt=null), Rental(id=RentalId
 (value=7e452a9f-51f2-4434-b4f8-bcb790641c77), book=Book(id=BookId
 (value=3e796d70-be9c-4b8f-aba0-8655bc6eb82f), title=Fables for Robots, author=Stanisław Lem), reader=Reader
 (id=ReaderId(value=bb61cd00-05f9-4006-a458-9a93ae33d133), firstName=Toby, lastName=Jones),
 rentedAt=2026-10-25T14:28:42.478919Z, returnedAt=null)]
```

Rysunek 13: Wyświetlanie wszystkich wypożyczeń dla czytelnika.

5. Literatura.

[1] „Dokumentacja języka Kotlin”. Dostęp: 23 października 2026. [Online]. Dostępne na: https://kotlinlang.org/docs/home.html